2. A Toy Calculator

We will start with a very simple language of expressions. The reason to look at an extremely simple language is to *focus* our attention on the essential concepts and results first, and then "build out" the language by adding constructs, which will start with similar constructions that can be handled using the same concepts and principles illustrated in the simple language, and then expanding the language as we introduce more complex and interesting concepts and principles.

The idea of keeping things simple ("Keep it simple, stupid!" or KISS) and focussing on the essential concepts is a philosophical idea called **"Occam's Razor"**, which recommends searching for explanations constructed with the smallest possible set of elements. This "principle of parsimony" is attributed to by a Franciscan friar (monk) and *Scholastic* philosopher named William of Ockham: *Entia non sunt multiplicanda praeter necessitatem*, which translates as "Entities must not be multiplied beyond necessity". For those interested, a character modelled on William of Ockham appears in Umberto Eco's 1980 novel *Il nome della rosa* (The Name of the Rose), a historical murder mystery set in an Italian monastery in the year 1327. The novel was made into a very successful movie The Name of the Rose (1986). The part of Brother William of Baskerville was played by actor Sean Connery.

Abstract syntax focusses on the essential structure of a language, whereas concrete syntax is a specification of the actual manifestation as text (or in some modern presentations, 2-dimensional graphical representations) of *phrases* of the language. We will revisit concrete syntax later, after starting our discussion with Abstract Syntax.

2.0 Abstract Syntax

Let us consider a toy language the *abstract syntax* of which consists of expressions in an inductively defined set Exp that is built up using the following three constructions:

1. (Base case: Integer numerals) $n \in Exp$.
   (Note: we shall use the terminology "numeral" for a syntactic representation of the mathematical notion of a *number*).
2. (Inductive case: plus) The sum of two expressions $e_1 + e_2 \in Exp$.
3. (Inductive case: times) The product of two expressions $e_1 + e_2 \in Exp$.

This inductive definition is written using an *abstract grammar* as follows:

$$e, e_1, e_2, \ldots \in Exp ::= n \mid e_1 + e_2 \mid e_1 * e_2$$

This abstract grammar can be represented in OCaml as a data type as follows:

```
type exp = Num of int
         | Plus of exp * exp
         | Times of exp * exp
;;
```

There are three constructors: `Num`, `Plus` and `Times` which label the three cases.

PROLOG Representation
We choose the constructor `numeral(N)` to be the PROLOG *coding* for the numeral N that is (in turn) a representation of the integer n.

We choose the constructor `plus(E1,E2)` to be the PROLOG *coding* for the expression $E1 + E2$ (that is a representation of the mathematical expression $e_1 + e_2$).

We choose the constructor `times(E1,E2)` to be the PROLOG *coding* for the expression $E1 * E2$ (that is a representation of the mathematical expression $e_1 \times e_2$.)

The elements of abstract syntax are tree-shaped, if one looks at their essential structure. Such *Abstract Syntax Trees* (ASTs for short) are a (very!) major concept in the specification of programming languages, their design, semantics and implementation, since they focus on the essential structure of programs, not their "surface" features.

Some example ASTs in our toy language can be defined as follows:

```
let t1 = Num 3;;
let t2 = Num 4;;
let t3 = Num 5;;
let t4 = Num 6;;

let t5 = Plus(t1, t2);;
let t6 = Times(t3, t4);;
let t7 = Times(t1, t2);;
let t8 = Plus(t3, t4);;

let t9 = Times(t5, t6);;
let t10 = Plus(t6, t7);;
let t11 = Plus(t1, t8);;
let t12 = Times(t7, t3);;
```

In what follows, we will write $e_1 \equiv e_2$ to mean that the ASTs e_1, e_2 are *identical* in all respects: node for node, label for label.

We can immediately define some useful functions on the structure of the ASTs. First, we can count the number of "nodes" in an AST. For the toy language, given above, here is a function that implements this:

```
let rec size e = match e with
    Num _ -> 1
  | Plus(e1, e2) -> 1 + (size e1) + (size e2)
  | Times(e1, e2) -> 1 + (size e1) + (size e2)
;;
```

You can check the type of function `size`, and that it returns the expected number of nodes for all the example trees given above.

Similarly, we can define the height of an AST as the function `ht`, as follows:

```
let rec ht e = match e with
    Num _ -> 0
  | Plus(e1, e2) -> 1 + (max (ht e1) (ht e2))
```

```
  | Times(e1, e2) -> 1 + (max (ht e1) (ht e2))
;;
```

You can check the type of function `ht`, and that it returns the expected height for all the example trees given above.

Exercise 2.1: Define a function `posttrav: exp -> string`, which given an AST *e* in type `exp`, returns a string that is the post-order traversal of the tree, that is the symbol at a node is presented *after* those of the left subtree and the right subtree are. You may use the function `Int.to_string` for rendering an integer as a string, and the strings "`+`" and "`*`" to represent `Plus` and `Times`. The caret symbol `^` allows the catenation of strings. (Note that in a post-order traversal, one need not use parentheses and commas as delimiters — spaces suffice.)

2.1 Toy Denotational Semantics

We will first specify a *mathematical evaluator* as a function (i.e., a functional relation) between expressions in *abstract syntax* and their mathematical meaning (*semantics*) as integers. Let us follow a convention that the syntactic elements are usually presented in green but underlined in places where the colour cannot be shown properly. In the presentation below, we will follow the convention of having the numeral be written as $\underline{n}$ whereas its integer *value* will be written n in blue.

The functional specification of the evaluator, a prototypical "*definitional interpreter*", is given as a mathematical function $eval : Exp \rightarrow \mathbb{Z}$. This function is recursive, and defined following the inductive structure of (the abstract syntax of) expressions. In specifying the function, we follow the now-standard convention of delimiting the syntactic expression (which I would normally highlight in green) by enclosing it in special brackets $[\![\quad]\!]$. The operators $+$ and $\times$ in the right-hand side of the equations in the definition below are *integer addition* and *multiplication*.

$$eval[\![\underline{n}]\!] = n$$

$$eval[\![e_1 \underline{+} e_2]\!] = eval[\![e_1]\!] + eval[\![e_2]\!]$$

$$eval[\![e_1 \underline{*} e_2]\!] = eval[\![e_1]\!] \times eval[\![e_2]\!]$$

It should be no surprise that we can easily code this evaluation function in OCaml:

```
let rec eval e = match e with
    Num n ->  n
  | Plus(e1, e2) ->  (eval e1) + (eval e2)  (* add *)
  | Times(e1, e2) -> (eval e1) * (eval e2)  (* multiply *)
;;
```

Note that we had used the expedient of representing the numeral $\underline{n}$ in our OCaml definition of type `exp` as `Num(n)` where `n` is of type `int`. This is because we are using OCaml as both a representational language for expressing ASTs as well as a *Meta-Language* for modelling arithmetic and writing our definitional interpreter.

There are other ways to have represented numerals, but we have chosen a representation that is convenient for us, even if it may seem that we have muddied the lines between representation (syntax) and its interpretation by using the same language. For this reason, we have written the ASTs in green, highlighting their occurrence with a lighter shade of green, while writing/highlighting in blue all the places where OCaml is used as a language for modelling arithmetic.

You can check the type of function `eval`, and that it returns the expected value for all the example trees given above.

The most important thing to notice about the function `eval` is that it provides meaning to an AST in terms of the meanings of its subtrees, i.e., follows the inductive structure of ASTs. This is a *Principle of Compositionality ("the meaning of the whole is obtained from the meanings of its parts in a well-defined way")* that is attributed, among others, to the logician Gottlob Frege (1879). Barbara Partee: "The meaning of a compound expression is a function of the meanings of its parts and of the way they are syntactically combined."

You should also notice the common shape of the definitions of `size`, `ht` and `eval`. They all are defined as recursive functions from the data type `exp` to an appropriate type (`int`) using case analysis on the AST, and recursive calls on the subtrees.

Programming Exercise (Solved below): Try to code the *eval* function in PROLOG as a predicate `eval(E,V).`

> When coding this function in PROLOG, which can also be used as a "meta-language" for defining the syntax and semantics of our toy language, we will exploit its computational features, in particular pattern-matching. Relational specifications are fairly easy to encode in PROLOG. Functional specifications are special cases, in which the function is expressed as a relation. Coding the definitional evaluator in PROLOG involves writing out the function *eval* in a relational form — where a function of k arguments is represented as a $(k + 1)$-ary relation, with the last position representing the result.

2.2 Operational Semantics

A *calculator* is an program that operates entirely on *representational forms, i.e.,* (abstract) syntax. That is, it is an algorithm whose data structures are the abstract syntax of the language, and it takes in an expression, and after performing a computation on a hypothetical machine returns as its answer a (canonical) expression that *represents* the value of the input expression. Let $Ans \subset Exp$ be this canonical set of representations. For the toy language, we let $Ans = Num$, the set of numerals.

Let us *specify* a simple calculator using "*inference rules*". These rules are to be read as "if all the antecedent statements of the rule hold, then the consequent statement holds". Some rules have no antecedents. These are called "*axioms*". Some rules may have "*side conditions*" or "*provisos*", which must hold in order for the rule to apply. In some treatments, the provisos are included as an antecedent. However, we prefer to write them alongside the rules — partly because we usually are defining a recursive relation, and will use induction as a proof technique. Note that induction and recursion are flip sides of the same coin — if a set is defined inductively, many useful functions *from* that set are recursively defined.

We define a relation $\Longrightarrow \subseteq Exp \times Ans$, which given an expression $e \in Exp$, returns a *numeral* $\underline{n}$ which is intended to be a valid representation of the *number* $eval[\![\ \underline{n}]\!]$.

(CalcNum) $\dfrac{}{\underline{n} \Longrightarrow \underline{n}}$

(CalcPlus) $\dfrac{e_1 \Longrightarrow \underline{n_1} \quad e_2 \Longrightarrow \underline{n_2}}{e_1 + e_2 \Longrightarrow \underline{n}}$ provided $pLUS(\underline{n_1}, \underline{n_2}, \underline{n})$

assuming we have an algorithm *pLUS* (a circuit in our hypothetical machine) that takes the numerals $\underline{n_1}, \underline{n_2}$ representing of two numbers, and returns a numeral $\underline{n}$ that represents their sum.

(CalcTimes) $\dfrac{e_1 \Longrightarrow \underline{n_1} \quad e_2 \Longrightarrow \underline{n_2}}{e_1 * e_2 \Longrightarrow \underline{n}}$ provided $tIMES(\underline{n_1}, \underline{n_2}, \underline{n})$

assuming we have an algorithm *tIMES* (a circuit in our hypothetical machine) that takes the numerals $\underline{n_1}, \underline{n_2}$ representing of two numbers, and returns a numeral $\underline{n}$ that represents their product.

When specifying the rules, we write them in a top-down manner: "*if* expression e_1 calculates to some numeral $\underline{n_1}$ and *if* expression e_2 calculates to some numeral $\underline{n_2}$, *then* expression $e_1 + e_2$ calculates to a numeral $\underline{n}$, *provided* the addition algorithm *pLUS* adds numerals $\underline{n_1}, \underline{n_2}$ to yield numeral $\underline{n}$".

However, when using the rules to reason about the calculation of the answer for a given expression, we may read the inference rules in a bottom-to-top manner: for example, the **(CalcPlus)** rule can be read as "suppose we are given expression $e_1 + e_2$; it calculates to a numeral $\underline{n}$ *if* expression e_1 calculates to some numeral $\underline{n_1}$ *and* expression e_2 calculates to some numeral $\underline{n_2}$, *and* the addition algorithm *pLUS* adds numerals $\underline{n_1}, \underline{n_2}$ to yield numeral $\underline{n}$".

[Note: This style of specification is called "*operational*" since it specifies the operational behaviour of the program. Within the operational style, this style is called "*Big-step*" or "Kahn-style" semantics (after the French computer scientist Gilles Kahn) since the computation is specified as giant steps relating inputs to outputs. Note that in this style of specification, if for an input there is no output (e.g., if the program goes into an infinite loop or if it "hangs", no corresponding input-output pair is specified.]

2.2.1 The Calculator Specification in PROLOG

The calculator can be coded in PROLOG as a predicate `calculate(E, A)`

```
/* A tiny integer calculator */
calculate(numeral(N),numeral(N)) :- integer(N).
calculate(plus(E1,E2), numeral(N)) :-
                          calculate(E1, numeral(N1)),
```

```
                            calculate(E2, numeral(N2)),
                            N is N1+N2.
calculate(times(E1,E2), numeral(N)) :-
                            calculate(E1, numeral(N1)),
                            calculate(E2, numeral(N2)),
                            N is N1*N2.
/*  A test case
eval( plus(
            times(numeral(3), numeral(4)),
            times(numeral(5), numeral(6))
          ),
     V).
*/
```

2.3 Correctness of the Operational Specification

Note that the calculator, as specified, is not guaranteed to be a function, whether partial or total. The functional nature of this relation is a property that we need to establish.

If (and indeed it is so) every numeral $\underline{n}$ corresponds to a unique integer n, and also that the circuits for *pLUS* and *tIMES* are (total) functions (the OCaml and PROLOG implementations of addition and multiplication are hopefully so, i.e., for every pair of representation of integers $\underline{n_1}, \underline{n_2}$ — represented in OCaml as `Num(n1)` and `Num(n2)`, and in PROLOG as `N1` and `N2` — both `n1+n2` and `n1*n2` in OCaml return unique integer representations (and likewise `N1+N2` and `N1*N2` each return unique PROLOG integer representations), then the relation $\Longrightarrow \subseteq Exp \times Ans$, i.e., the `calculates` relation, is indeed a (total) function.

Nor is it clear that *any* answer given by the calculator specification will indeed be a valid representation of the *mathematical value* to which any expression will *eval*. However, if it does not do so, then our calculator is worthless, a junk specification, and we should throw away any implementation of this specification. For the correctness of the specification we require that the circuits $pLUS(\underline{n_1}, \underline{n_2}, \underline{n})$ and $tIMES(\underline{n_1}, \underline{n_2}, \underline{n})$ — and their corresponding OCaml and PROLOG analogues `N1+N2` and `N1*N2` — are *semantically correct*, i.e., they are (total) functions that return *answers* that are indeed the mathematical representations of $n_1 + n_2$ and $n_1 * n_2$.

Theorem 2.0 (Soundness of $\underline{e} \Longrightarrow \underline{n}$, i.e., of `calculate(E, A)`).
Assume that (i) every numeral $\underline{n}$ corresponds to a unique integer $n \in \mathbb{Z}$, and (ii) for every pair of numerals $\underline{n_1}, \underline{n_2}$, (ii-a) if $pLUS(\underline{n_1}, \underline{n_2}, \underline{n})$ then $eval[\![\underline{n}]\!] = eval[\![\underline{n_1}]\!] + eval[\![\underline{n_2}]\!]$, and (ii-b) if $tIMES(\underline{n_1}, \underline{n_2}, \underline{n})$ then $eval[\![\underline{n}]\!] = eval[\![\underline{n_1}]\!] \times eval[\![\underline{n_2}]\!]$.

Then for all $\underline{e} \in Exp, \underline{n} \in Ans$, if $\underline{e} \Longrightarrow \underline{n}$ then $eval[\![\underline{e}]\!] = eval[\![\underline{n}]\!]$.

[In terms of the PROLOG encoding, if `calculate(E, A)` then

`A = numeral(N),` where `N` is the representation of $eval[\![e]\!]$

The proof is by induction on the structure of the (abstract syntax of) expressions $e \in Exp$. This is because Exp is defined by induction, and also because the relation $e \Longrightarrow n$ is defined by inductive case analysis on $e \in Exp$

Proof. By structural induction on $e \in Exp$.
Assume $e \Longrightarrow n$.

Base Case: $ht(e) = 0$. By case analysis, $e \equiv n'$ for some numeral n'
By (**CalcNum**) $n' \Longrightarrow n'$. So $n \equiv n'$. So $e \equiv n$
Trivially, $eval[\![e]\!] = eval[\![n]\!]$.

Induction Hypothesis (IH): Assume that for all $e' \in Exp, n' \in Ans$ such that $ht(e') \leq k$, if $e' \Longrightarrow n'$ then $eval[\![e']\!] = eval[\![n']\!]$.

Induction Step: Consider $e \in Exp$ such that $ht(e) = k+1$.
By analysis, either (i) $e \equiv e_1 + e_2$ or (ii) $e \equiv e_1 * e_2$.

(i) $e \equiv e_1 + e_2$, where $ht(e_1) \leq k$ and $ht(e_2) \leq k$.
Suppose $e \Longrightarrow n$.
Then by (**CalcPlus**) $e_1 \Longrightarrow n_1$, $e_2 \Longrightarrow n_2$ for some numerals n_1, n_2.
By IH on e_1, $eval[\![e_1]\!] = eval[\![n_1]\!]$ and by IH on e_2, $eval[\![e_2]\!] = eval[\![n_2]\!]$
Recall that by definition, $eval[\![e_1 + e_2]\!] = eval[\![e_1]\!] + eval[\![e_2]\!]$
Thus from the definition of ***eval*** and the IHs,
$eval[\![e_1 + e_2]\!] = eval[\![n_1]\!] + eval[\![n_2]\!]$
By the condition $pLUS(n_1, n_2, n)$ and the assumption on $pLUS$ that
$eval[\![n]\!] = eval[\![n_1]\!] + eval[\![n_2]\!]$, we have $eval[\![e]\!] = eval[\![n]\!]$.

(ii) $e \equiv e_1 * e_2$. Similar to (i). [**Exercise**: Work out this case] □

Note that **Soundness** only says that *if* the calculator returns an answer, *then* the answer is semantically correct. However, there is no guarantee that it does indeed return an answer on any given input expression. So a calculator that does not return any result is trivially sound! But probably not what you want. One can also adapt the soundness theorem to also show that the calculator specification is indeed a *total function*.

The converse of soundness, called "*Completeness*", may be somewhat harder to state and prove (especially as the language gets more complicated)... and indeed *does not hold of actual calculators* because of the finiteness limitations of the numerals which we can actually write or with which we can compute. However, for the present specification, we can show that *in principle* (that is if we have enough time and space for the representation of even very large numbers and also circuits that can add or multiply them) the calculator will return an answer that is a correct representation of the meaning to which any express evaluates.

Theorem 2.1 (**Completeness** of $\underline{e} \Longrightarrow \underline{n}$, i.e., of `calculate(E, A)`).

Assume that (i) every integer $n \in \mathbb{Z}$ can be represented as a numeral $\underline{n}$; (this numeral need not be unique).
(ii) for every pair of numbers n_1, n_2 represented by numerals $\underline{n_1'}, \underline{n_2'}$, (ii-a) the mathematical integer $n_1 + n_2$ is represented by some numeral $\underline{n'}$ such that $pLUS(\underline{n_1'}, \underline{n_2'}, \underline{n'})$, and (ii-b) the integer $n_1 \times n_2$ is represented by some numeral $\underline{n'}$ such that $tIMES(\underline{n_1'}, \underline{n_2'}, \underline{n'})$.

Then for all $\underline{e} \in Exp$, and $n \in \mathbb{Z}$, if $eval[\![\ \underline{e}\ ]\!] = n$ then $\underline{e} \Longrightarrow \underline{n'}$ for some numeral $\underline{n'}$ such that $eval[\![\ \underline{n'}\ ]\!] = n$

[In terms of the PROLOG encoding, if $eval[\![\ \underline{E}\ ]\!] = n$ then `calculate(E, numeral(N))` where $eval[\![\ \underline{N}\ ]\!] = n$.]

The proof is by induction on the structure of the (abstract syntax of) expressions $\underline{e} \in Exp$. This is because Exp is defined by induction, and also because the recursive function $eval[\![\ \underline{e}\ ]\!]$ is defined by inductive case analysis on $\underline{e} \in Exp$.

Proof. By structural induction on $\underline{e} \in Exp$.
Assume $eval[\![\ \underline{e}\ ]\!] = n$

Base Case: $ht(\underline{e}) = 0$. By case analysis, $\underline{e} \equiv \underline{n''}$ for some numeral $\underline{n''}$.
By (**CalcNum**) $\underline{n''} \Longrightarrow \underline{n''}$. We let $\underline{n'} \equiv \underline{n''}$ $(\equiv \underline{e})$.
Trivially, $\underline{e} \Longrightarrow \underline{n'}$ and $eval[\![\ \underline{e}\ ]\!] = eval[\![\ \underline{n'}\ ]\!] = n$.

Induction Hypothesis (IH): Assume that for all $\underline{e'} \in Exp$ such that $ht(\underline{e'}) \leq k$, for all $n' \in \mathbb{Z}$, if $eval[\![\ \underline{e'}\ ]\!] = n'$ then $\underline{e'} \Longrightarrow \underline{n''}$ for some numeral $\underline{n''}$ such that $eval[\![\ \underline{n''}\ ]\!] = n'$

Induction Step: Consider $\underline{e} \in Exp$ such that $ht(\underline{e}) = k+1$.
By analysis, either (i) $\underline{e} \equiv \underline{e_1} + \underline{e_2}$ or (ii) $\underline{e} \equiv \underline{e_1} * \underline{e_2}$.

(i) $\underline{e} \equiv \underline{e_1} + \underline{e_2}$, where $ht(\underline{e_1}) \leq k$ *and* $ht(\underline{e_2}) \leq k$.
Suppose $eval[\![\ \underline{e}\ ]\!] = n$
Then by the recursive definition of $eval[\![\ \underline{e}\ ]\!]$,
$eval[\![\underline{e_1} + \underline{e_2}]\!] = eval[\![\underline{e_1}]\!] + eval[\![\underline{e_2}]\!]$
Let $eval[\![\ \underline{e_1}\ ]\!] = n_1$ and $eval[\![\ \underline{e_2}\ ]\!] = n_2$, i.e., $n_1 + n_2 = n$
By IH on $\underline{e_1}, \underline{e_2}$ respectively, $\underline{e_1} \Longrightarrow \underline{n_1'}$, $\underline{e_2} \Longrightarrow \underline{n_2'}$ for some numerals $\underline{n_1'}, \underline{n_2'}$, such that $eval[\![\ \underline{n_1'}\ ]\!] = n_1$ and $eval[\![\ \underline{n_2'}\ ]\!] = n_2$
Now by assumption about $pLUS$ that the integer $n_1 + n_2$ is represented by some numeral $\underline{n'}$ such that $pLUS(\underline{n_1'}, \underline{n_2'}, \underline{n'})$, we have by the rule (**CalcPlus**) (read top-down) that $\underline{e} \Longrightarrow \underline{n'}$ where $eval[\![\ \underline{n'}\ ]\!] = n_1 + n_2 = n$.

(ii) $\underline{e} \equiv \underline{e_1} * \underline{e_2}$. Similar to (i). [**Exercise**: Work out this case] □

2.4 A Toy Compiler for the Toy Language

The calculator specification is an *input-output* relational specification of an interpreter. An alternative model of program execution is to *compile* the program into a sequence of "*op-codes*" for an *abstract machine*. The abstract machine that we have in mind is a simple stack-based evaluator, where the main data structure is a *stack* on which answers are pushed, or from which the operands of an operator are popped and replaced by the result of the operation. The configurations of the abstract machine are of the form $\langle s, c \rangle$ where s is a stack of answers, and c is a sequence of op-codes.

Let us first define the op-codes of the machine and their intended operation:

1. *LD*(n) - which loads the specified constant value n onto the stack
2. *PLUS* - which pops the top two values from the stack, adds them and pushes the resulting value back onto the stack;
3. *TIMES* - which pops the top two values from the stack, multiplies them and pushes the resulting value back onto the stack.

In OCaml, we define a data type of opcodes:

```
type opcode = LD of int | PLUS | TIMES;;
```

In PROLOG these opcodes may be represented as:

```
ldop(N), plusop, and timesop
```

First some preliminaries about lists and a relational encoding in PROLOG of the append function:

```
append([], L, L).
append([X|R], L, [X|Z]):- append(R, L, Z).
```

This code may be read as follows: (1) appending an empty list to any list `L` results in the list `L`. (2) appending a non-empty list, whose first element is `X` and the rest of the list is `R`, to a list `L` results in a non-empty list whose first element is `X`, and the rest of the list is `Z`, which is the result of appending `R` to `L`.
Note that PROLOG does not allow nested recursive calls, but instead employs recursive calls on the right side of the `:-` symbol, and the chaining of outputs to inputs by having the named output of one predicate to be an input to another.

The *compile* function takes an expression $e \in Exp$ and returns a *list* or sequence (of op-codes:

```
let rec compile e = match e with
   Num(n) -> [  LD(n) ]
|  Plus(e1, e2) -> (compile e1) @ (compile e2) @ [ PLUS ]
|  Times(e1, e2) -> (compile e1) @ (compile e2) @ [ TIMES ]
;;
```

Compiling a numeral n results in a *singleton* list consisting of the op-code `LD(n)` Compiling an expression $e_1 + e_2$ results in a sequence of op-codes that first has the op-codes sequence `C1` obtained by compiling expression e_1 followed by the sequence `C2` obtained from compiling e_2, followed by the singleton list containing the op-code `PLUS`. The code for $e_1 * e_2$ is similar, but with opcode `TIMES` instead.

The relational encoding `compile(E,C)` of the *compile* function is quite similar in structure to the relational specification for `calculate(E,A)`.

```
/* A simple compiler for the toy language */

compile(numeral(N),[ldop(N)]) :- integer(N).
compile(plus(E1,E2),C ) :-
                          compile(E1, C1),
                          compile(E2, C2),
                          append(C1, C2, C3),
                          append(C3, [plusop], C).
compile(times(E1,E2), C) :-
                          compile(E1, C1),
                          compile(E2, C2),
                          append(C1, C2, C3),
                          append(C3, [timesop], C).
```

Compiling a numeral N results in a *singleton* list consisting of the op-code `ldop(N)`
Compiling an expression $E_1 + E_2$ results in a sequence of op-codes that first has the op-codes sequence `C1` obtained by compiling expression E_1 followed by the sequence `C2` obtained from compiling E_2, followed by the singleton list containing the op-code `plusop`. The code for $E_1 * E_2$ is similar, but with `timesop` instead.
Note the similarity in structure between the predicates `compile(E,C)` and `calculate(E, A)`.

2.4.1 The Abstract Machine

Let us first specify the one-step transitions of the stack machine, writing [ ] for the empty list and $x :: l$ for a non-empty list with initial element x and remaining list l. Note that the same notation works for a stack.

(Load) $\langle s,\ LD(\underline{n}) :: c\rangle \gg\!\!\Rightarrow \langle \underline{n} :: s, c\rangle$
(Plus) $\langle \underline{n_2} :: \underline{n_1} :: s,\ PLUS :: c\rangle \gg\!\!\Rightarrow \langle \underline{n} :: s, c\rangle$ where $pLUS(\underline{n_1}, \underline{n_2}, \underline{n})$
(Times) $\langle \underline{n_2} :: \underline{n_1} :: s,\ TIMES :: s\rangle \gg\!\!\Rightarrow \langle \underline{n} :: s, c\rangle$ where $tIMES(\underline{n_1}, \underline{n_2}, \underline{n})$

Note that the machine cannot make a transition if there are *no* op-codes that remain to be executed. In this case, the machine halts, and we can "unload" the answer at the top of the stack (assuming the stack is not empty; otherwise it is an error).

The machine gets "stuck" (also an error) if the op-code to be executed is either *PLUS* or *TIMES*, but there *aren't* at least two values on the stack for the arithmetic operation.

The OCaml encoding of the stack machine is a *tail-recursive function* `stackmc` that computes the (reflexive) transitive closure of the single step transitions specified above: for any stack machine configuration stackmc is recursively called
If the execution reaches a configuration where it gets stuck, an exception is raised.
If it finishes executing all its opcodes, and has a single answer on stack, that answer is unloaded.

```
exception Stuck;;

let rec stackmc s c = match s,c with
    s', LD(n)::c' ->  stackmc ((Num n)::s') c'
```

```
  | (Num n2)::(Num n1)::s', PLUS::c' ->
                        stackmc ((Num (n1+n2))::s') c'
  | (Num n2)::(Num n1)::s', TIMES::c' ->
                        stackmc ((Num (n1*n2))::s') c'
  | a::[ ], [ ] -> a          (* No more opcodes *)
  | _,_ -> raise Stuck      (* Anything else *)
;;
```

The encoding in PROLOG combines the single transitions indicated above with a *tail-recursive call* to execute from the resulting state. The predicate `stackmc(S, C, A)` is to be read as "executing the stack machine from a state $\langle S, C \rangle$ on the LHS of a single transition results in unloading an answer `A` if executing the machine from the state $\langle S', C' \rangle$ on the RHS of that transition results in unloading that answer `A`.

[Note the use in PROLOG of the idiom "`N is N1 + N2`" to force the computation of the arithmetic expression. See what happens if you omit this PROLOG phrase and instead write `N1 + N2` in place of `N` on the stack.]

```
/* A stack machine: A simple abstract machine */


stackmc( [A|S1], [ ], A).
stackmc( S, [ldop(N)|C], A) :-
                              stackmc([numeral(N)|S], C, A).
stackmc( [numeral(N2)|[numeral(N1)|S]], [plusop|C], A ) :-
                            N is N1+N2, stackmc([numeral(N)|S], C, A).
stackmc( [numeral(N2)|[numeral(N1)|S]], [timesop|C], A ) :-
                            N is N1*N2, stackmc([numeral(N)|S], C, A).


/*  Suggested test query:
   compile(
            plus( times(numeral(3), numeral(4)),
                  times(numeral(5), numeral(6)) ),
            C),
stackmc( [ ], C, A).

*/
```

The suggested test first compiles an expression into an op-code sequence, and then executes this op-code sequence starting with the empty stack.

2.4.2 Correctness of the Abstract Machine model

Proposition 2.2 (Correctness of compile-execute)
For all $\underline{e} \in Exp, \underline{n} \in Ans$ $\langle [], compile(\underline{e}) \rangle >\!\!\Longrightarrow^* \langle \underline{n} :: s, [] \rangle$ if and only if $\underline{e} \Longrightarrow \underline{n}$ (for some stack s)

Ideally we should be able to prove this result using induction on the structure of $\underline{e}$, since both $\underline{e} \Longrightarrow \underline{n}$ and the function *compile* are defined accordingly. One technical issue is that the execution rules of the abstract machine are not. Instead, the machine execution rules are by case analysis on the first op-code in the sequence.

Moreover, it is also hard to apply the induction hypothesis (IH) if in the statement of the result, the stack in the LHS state and the op-code list in the RHS state are to be empty. (**Exercise**: Why?)

Fortunately, there is a very strong isolation (modularity) in how the abstract machine executes: the execution of the code obtained from *compile*($\underline{e}$) *never accesses the stack below what it places there,* and execution consumes the op-codes *serially*.

Hence we need to generalise the statement of the result to the (much stronger) result:

Theorem 2.3 (Correctness of compile-execute)
For all $\underline{e} \in Exp, \underline{n} \in Ans,$ for *all* stacks s' and op-code lists c':
$\langle s', compile(\underline{e}) @ c' \rangle \gg\!\!\Longrightarrow^* \langle \underline{n} :: s', c' \rangle$ *if and only if* $\underline{e} \Longrightarrow \underline{n}$

Solution: A possible PROLOG coding of the *eval* function. Note the similarity and yet a major difference from the `calculate(E, A)` predicate — the result of evaluation is not a numeral but a (PROLOG) integer value.

```
/* A definitional interpreter for the toy language */
eval(numeral(N),N) :- integer(N).
eval(plus(E1,E2), N) :-
                          eval(E1, N1),
                          eval(E2, N2),
                          N is N1+N2.
eval(times(E1,E2), N) :-
                          eval(E1, N1),
                          eval(E2, N2),
                          N is N1*N2.



/*  A test case
calculate( plus(
            times(numeral(3), numeral(4)),
            times(numeral(5), numeral(6))
                ),
          N).
*/
```

24.1.2025 Quiz 4: Abstract syntax trees

Q1 [5 marks]

Write an OCaml program `subtrees: exp -> (exp list)` which, given an expression *e*, returns a list of all subtrees of *e* (including *e* itself).

```
let rec subtrees e = match e with
   Num(n) -> e :: [ ]
|  Plus(e1, e2) ->  e :: (subtrees e1) @ (subtrees e2)
|  Times(e1, e2) -> e ::(subtrees e1) @ (subtrees e2)
;;
```